The next question is, are you going to write any tests specifically to target the logic in the controller? And the answer, perhaps surprisingly, is "no." Or at least "not with the tools already discussed."

Controller tests, which have been a feature of Rails from the beginning, have increasingly been seen as lower-value tests in a couple of different ways.

If you write Rails applications the way that the Rails core team writes Basecamp (their flagship application) and follow the process and structures as espoused by Rails founder David Heinemeier Hansson, then regardless of how your controllers are structured, controller tests feel like duplicates of integration tests. Since there has been a lot of work in Rails 5 to make integration tests faster, controller tests become less valuable.

If you do more aggressive test-driven development, or you like moving application logic to separate workflow objects, then the controller doesn't have much functionality of its own, and you're concerned with a controller test duplicating model or workflow tests.

In either case, it's tricky to write a controller test that covers the controller logic without duplicating other tests. When I talk about test doubles in Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129, I'll discuss a strategy that might help. Also, Rails and RSpec still provide some features for testing controllers, which I'll discuss in Chapter 11, *Testing Rails Display Elements*, on page 217.

Since you've put the business logic in the workflow object, the controller doesn't have much logic, but it does have some. Specifically, the controller sends data both to the workflow object and onward to the view layer. In the current test structure, these functions are controlled by the system test. Notice that responsibilities are separated here—almost nothing that the controller does is dependent on the logic of creating and saving projects.

Although I haven't stressed the point, the controller also needs to do something in case the action object errors or does something else unexpected. I don't normally like to test error conditions in system tests—too slow. I try to create the error conditions at the unit level.

The code to clear the controller-specific lines in the integration test looks like:

test_first/03/app/controllers/projects_controller.rb

```
class ProjectsController < ApplicationController
  def new
    @project = Project.new
  end
```